

Sweetgreen: RPA-Candidate Scan

Automate the rule, route the exception: where a bot belongs on Sweetgreen's order-and-payment flow

IE-GY 9113B • Systems Integration: From ERP to Agentic AI

Week 4 • Board Memo §4 (Automation Journey) • Storyboard S6 • Milestone 1: Diagnose

[illegible]

Matches the control register and diagnostic points in the Lab 1 process map and the Week 4 RPA-candidate scan — a mirror, not a template.

Two candidates, two tools

Repetitive + rules-based + stable, or it isn't a candidate. Both of Sweetgreen's diagnostic points from Lab 1 pass, into different tools.

D2 — Feasible? (live vs. cached availability)

→ RPA • Lab 2 (Make)

- Repetitive: every order passes through it
- Rules-based: approve when live and cached availability agree and the customization is supported, else hold (complete once the freshness gate is folded in)
- Stable: the rule and its two source systems (Olo, Crunchtime) rarely change
- Thread: Operations primarily, also Cash and Customer
- Prize: 6 of 30 synthetic orders (20%) hit the mismatch, \$44 modeled exposure, 18.7 min late on average
- Unattended on the routine path; human stays at the exception queue, every time the gate disagrees

D3 — Recovery agreed? (staff assess options)

→ AI agent • Lab 3, Week 9

- Repetitive: every exception reaches it
- Rules-based: no. Two reasonable staff members could offer different resolutions, depending on urgency, prep ETA, and consent
- This is the judgment step: an assistant can recommend a recovery option for manager review, but can't make the customer's choice
- Not this week's build target. Previewed now, built in Lab 3 once intelligent automation (Week 7) and agent design (Week 8) are covered

What we rejected, and why

A scan that only says yes hasn't done the diagnostic work. Three steps, three different reasons to say no.

D1 — Order acceptance beyond payment

- Repetitive and probably stable, but the map doesn't yet state the full acceptance rule. Automating an underspecified rule risks locking in an implicit policy. That's a mapping gap, not an automation call yet.

Quality gate — does the meal match the order?

- Passes the rules test on paper, but fails the RPA pattern itself: the input is a physical, visually verified meal, not screen-readable system data. Kept as a routing decision, not elevated to a control point.

The stock-accuracy root cause behind the mismatch

- Automating around a stale cache (checking it faster) would pave the cow path. The fix is closing the sync gap between Olo and Crunchtime, a redesign question, not an RPA one.

D2, costed and governed

~10

staff-hours a day, order of magnitude, at an assumed 300 orders/store/day and a 20% mismatch rate. Not sourced, validate before board use

The number, order-of-magnitude

- 10 min manual reconciliation per affected order (60 min / 6 orders in the 30-order sandbox)
- Labor-removed side of payback; Make build + licence side is Lab 2's number, not yet known
- This is a shape, not a commitment. Validate against real per-store volume and mismatch rates

Attended or unattended?

Unattended, with a human checkpoint

- Unattended on the routine path, no human needed when live and cached availability agree
- Human stays exactly where the map already routes them: the exception queue, every time the gate disagrees
- Not case-by-case on the happy path, every time on the path that needs judgment

Human oversight & workforce note

A narrower claim than “redeployment”

- Every freshness disagreement routes to a person, never auto-resolved. Guessing which source is right is exactly the kind of confident-but-wrong answer a bot gives without knowing it's wrong
- This candidate removes a manual copy-and-compare check, not a role
- Freed minutes return to staff already doing exception handling
- A fuller workforce answer, as automation scales past this one candidate, is Board Memo \$7's job

WHERE THIS LANDS

The scan is the argument. The build is Lab 2.

Board Memo §4

Automation Journey, this scan is the evidence

Slide S6

Automation-opportunity slide, still inside Milestone 1: Diagnose

Lab 2, Week 6

Build the D2 freshness-and-feasibility gate in Make, then break it

Lab 3, Week 9

Build the D3 recovery-assist agent, previewed here

Full detail: wiki/project/week4/RPA-Candidate-Scan-Sweetgreen.md